

Lab 5 — Database Design from Requirements

Module II · Week 5 · Duration: 2 hours Tools: paper/diagram tool + PostgreSQL · Deliverables: design.md, schema.sql, data.sql, queries.sql

Objectives

Take a messy, natural-language specification and carry it all the way to a working database — the exact workflow from Lectures 13–15:

1. Extract **entities**, **attributes**, **relationships** from a narrative.
2. Draw an **ER diagram** with correct cardinalities and participation.
3. Apply the **ER → relational mapping** algorithm (Lecture 14).
4. Implement the schema in **PostgreSQL** with proper keys and constraints.
5. **Populate** it with sample data and run queries that prove it works.
6. Justify your design choices and spot potential **pitfalls** (Lecture 15).

Time budget (2 h): read + ER diagram ~35 min · mapping ~20 min · `schema.sql` ~30 min · `data.sql` ~20 min · `queries.sql` ~15 min · `design.md` write-up ~10 min (throughout).

Setup (5 min)

Confirm PostgreSQL is running (see `infrastructure.md`) and create a fresh database:

```
createdb lab5          # or: psql -c 'CREATE DATABASE lab5;'
psql lab5 -c '\conninfo' # confirm you're connected
```

You will build `schema.sql` and load it with:

```
psql lab5 -f schema.sql
psql lab5 -f data.sql
psql lab5 -f queries.sql
```

If PostgreSQL is unavailable, you may use SQLite (`sqlite3 lab5.db < schema.sql`) — but write **portable** SQL and note the substitution in `design.md`.

The specification — “MediTrack” clinic

MediTrack runs a network of **clinics**. Each clinic has a name, an address, and a phone number. **Patients** register once with the network (name, date of birth, blood group, and one or more phone numbers) and may visit **any** clinic.

Doctors work at exactly one clinic and have a specialization. A patient books an **appointment** at a clinic with a specific doctor for a date and time; the appointment records the reason for the visit and a status (booked / completed / cancelled).

During a completed appointment a doctor may issue a **prescription** listing one or more **drugs**, each with a dosage and duration. A drug has a name and a manufacturer. The same drug can appear on many prescriptions.

Each completed appointment generates one **bill** with an amount and a paid/ unpaid status. The system must answer questions like: *which doctors are busiest, which drugs are prescribed most, and how much revenue each clinic earned.*

Part 1 — Conceptual design (35 min)

1.1 List every **entity set** you find, with its attributes. Mark: - the **primary key** (introduce a surrogate id where no stable natural key exists), - any **multivalued** attribute (e.g. patient phone numbers), - any **derived** attribute.

1.2 List every **relationship**, and for each state its **mapping cardinality** (1:1 / 1:N / M:N) and **participation** (total / partial). Watch for: - **doctor works_at clinic** — what cardinality? - **patient books appointment** — is **appointment** better modelled as an **entity**? - **prescription contains drug** — why is this M:N with attributes?

1.3 Draw the **ER diagram** (any tool: dbdiagram.io, draw.io, ERDplus, or a clear photo of a hand drawing). Save it as **er.png** (or paste an ASCII/Mermaid sketch into **design.md**).

[Checkpoint] **Checkpoint:** you should have ≥ 6 **entity sets**, at least **one weak entity or first-class “appointment” entity**, at least **one M:N relationship with its own attributes** (prescription–drug), and every relationship labelled with cardinality **and** participation.

Part 2 — Logical design / mapping (20 min)

2.1 Apply the **7-step mapping algorithm** (Lecture 14). In **design.md**, write one line per relation of the form:

```
patient(patient_id PK, name, dob, blood_group)
patient_phone(patient_id FK, phone, PK(patient_id, phone))  -- multivalued attr
...
```

2.2 For **each** M:N relationship, confirm you created a **junction table** with a composite primary key. For each 1:N, confirm the foreign key sits on the **“many”** side.

2.3 Identify **one** potential **fan trap or chasm trap** in your design (e.g. reaching a patient’s drugs only through appointments) and explain in **design.md** how your schema avoids giving wrong answers.

Part 3 — Implement the schema (30 min)

Write **schema.sql**. Requirements:

- Correct **primary keys** and **foreign keys** for every table.
- NOT NULL on every attribute the spec says is required.
- At least **three CHECK constraints** (e.g. `status IN (...)`, `amount >= 0`, a valid `blood_group`).
- At least one **DEFAULT** (e.g. appointment status defaults to 'booked').
- Use portable types (INTEGER, TEXT, NUMERIC, DATE).

Load it and confirm no errors:

```
psql lab5 -f schema.sql
psql lab5 -c '\dt'      # list your tables
```

[Warning] **Order matters:** create referenced tables (e.g. **clinic**, **drug**) **before** the tables that reference them, or Postgres will reject the foreign keys.

Part 4 — Populate & query (35 min)

4.1 Write **data.sql** inserting **realistic** sample data: - ≥ 2 clinics, ≥ 4 doctors, ≥ 6 patients (one with **two** phone numbers), - ≥ 8 appointments across **both** clinics and **several statuses**, - ≥ 3 prescriptions

referencing ≥ 5 drugs, - bills for the completed appointments (mix of paid/unpaid), - **at least one patient with no appointments** (to test outer joins).

4.2 Write `queries.sql` answering these business questions — each preceded by a comment with its number:

1. All appointments at a given clinic on a given date, with patient and doctor names.
2. The **busiest doctor** (most completed appointments).
3. The **top 3 most-prescribed drugs** (by number of prescriptions they appear on).
4. **Revenue per clinic** from **paid** bills (include clinics with 0 — outer join).
5. Patients who have **never** had a completed appointment (`NOT EXISTS`).
6. For each patient, the **list of drugs** ever prescribed to them (join across appointment $\rightarrow$ prescription $\rightarrow$ drug).

[Checkpoint] **Checkpoint:** Q4 must show a clinic with **0** revenue if one had no paid bills (proves your outer join is correct). Q5 must return the no-appointment patient you inserted in 4.1.

Part 5 — Design review (throughout, write-up 10 min)

In `design.md`, add a short **design log**:

- **Two ambiguities** in the spec, the question you'd ask the client, and the assumption you made.
- **One alternative design** you rejected (e.g. storing drugs as a text list on the prescription) and **why** it's worse (which anomaly it causes).
- A one-line argument that your schema is at least in **3NF** (no obvious transitive dependencies) — full proof comes in Lab 6.

Stretch goals (optional)

- Add a **receptionist** entity who **schedules** appointments; decide if it's an aggregation over the booking relationship.
- Add an **ISA hierarchy**: `staff  $\rightarrow$  {doctor, receptionist}`; map it with Method A and note the trade-off.
- Add a **CHECK** that an appointment's `status='completed'` **requires** a bill (discuss why this may need a trigger instead).

Submission

Submit a folder `lab05/` containing:

- **design.md** — entity/relationship lists, the mapping, the design log, and either `er.png` referenced or an embedded diagram.
- **schema.sql** — the full DDL (loads with no errors on a fresh database).
- **data.sql** — your sample data.
- **queries.sql** — the six queries, each commented with its number and a one-line description.
- **er.png** (or a Mermaid/ASCII diagram inside `design.md`).

Self-check: on a brand-new database, `psql fresh -f schema.sql -f data.sql -f queries.sql` (or the SQLite equivalent) runs top to bottom with **no errors** and the checkpoint queries return the expected shapes.